AX

AXON-X · AXON-WATCH

How-To *Handbook*

Your field guide for bootstrapping, verifying, troubleshooting, and shipping thin slices in the next-generation operator console.

Shell :4173 | **Control plane** :8787 | **Watch** :8788

Last verified · 2026-07-22 — Gate 5 closed (DAG, conflict serialization, fan-out, replans, synthesis, sibling stream preservation); Gate 4 closed; Mission Control task board. Scheduler still **off** by default. After every push run `./scripts/ops/watch-fast-gate.sh`.
Generated 22 Jul 2026

CONSOLE-WEB | CONTROL-PLANE | SHARED-TYPES

FIELD GUIDE

16 chapters

23 Troubleshooting

24 Debugging playbook

25 Tips And Tricks

26 Snippet cookbook

27 Upgrading and updating

28 What A Good Next Slice Looks Like

29 Final Guidance

30 CI, merge workflow, and worker agents

31 or follow a known run id:

32 Typical Fast Gate early fail: file-size ratchets in scripts/guardrails/hotspot_budgets.json

33 CI locally

34 Open PR

35 PR checks

36 Worker scheduler tests

37 Stale reconcile (when loaded on CP)

38 Autonomy gates & service identity (operator directives)

39 ~/.config/axon-watch/deployment.env

40 writes ~/.config/axon-watch/mtls/{ca,client,server}.{crt,key}

41 then push and confirm Axon-X Fast Gate is green on GitHub

42 Recent operator features (Gate 3–5 + console UX)

43 From repo root — polls the latest Fast Gate run for this branch

44 Online research: SearXNG and legacy Google

45 then set AXON_WATCH_SEARXNG_URL=http://127.0.0.1:8080 in .env (or vault)

Read this first

Practical guide for operators, reviewers, and developers working in the **axon-watch** repo — the greenfield home of **Axon-X**.

- Plans live in `axon-local/Plans/Axon-Watch/` — implement here, don't redefine semantics there.
- Start with `./scripts/dev/up.sh`, verify with `npm run verify`.
- Treat bootstrap-real and feature-real as different things — see parity ledger before assuming parity.

01 Table of Contents

02 Handbook map

AUDIENCE	START HERE	THEN READ
Operator (daily use)	Quick Start	Runtime auth, CLI, and tools, VAXON Desktop, Operator manual
Teacher / reviewer	Teaching Axon-X	Verification, `docs/FINAL_PARITY_VERIFICATION.md`
Developer	Codebase in plain English	Source index, Common working patterns
Integrator / merge	CI, merge, and worker agents	`docs/CI_GATES.md` , `./scripts/ops/watch-fast-gate.sh`
Autonomy / remote host	Autonomy gates & service identity	Recent operator features
Debugger	Debugging playbook	Troubleshooting
Upgrader	Upgrading & updating	`./scripts/ops/sync_planning_mirror_to_axon_local.py`

Shorter onboarding: `docs/AXON-X-STARTER-GUIDE.md`

03 Quick Start

This section is the living operator onboarding guide.

Start the stack

Always-on host (this machine — preferred):

COMMAND	WHAT IT DOES
`axonhealth`	Probe console + control-plane + watch (+ key APIs)
`axonrestart`	Soft restart of systemd user units, then health check
`axonrevive`	**Use when the shell is empty** (Runtime unavailable / No workspace). Force-kills a wedged control-plane, restarts all three units, health-checks

```
axonhealth          # is everything up?
axonrevive          # empty shell / hung API — fix it
# then hard-refresh http://127.0.0.1:4173
```

These are on your PATH (`~/.local/bin` → `bin/` in this repo). See Snippet cookbook.

Dev bootstrap (alternate, not used when systemd owns the ports):

```
cd /home/edp/axon-nvme/repos/axon-watch
./scripts/dev/up.sh
./scripts/dev/check-health.sh
```

Open **http://127.0.0.1:4173** and hard-refresh after upgrades (`Ctrl+Shift+R`).

> **Important:** `./scripts/dev/down.sh` does **not** stop systemd always-on units. On this host use `axonrestart` / `axonrevive`.

Pick a real workspace

The left sidebar should show **axon-watch** and **axon-local** (bound project roots). Start with **axon-watch** for Axon-X development, or **axon-local** when you need the legacy repo context.

Demo names like `workspace_nlp` are mock catalog entries — they are hidden when real project bindings exist.

Two modes — different jobs

Axon-X has two layout modes (top-right toggle). They are **not** two different apps; they are two views over the same workspace, runs, and APIs.

	OPERATOR MODE	IDE MODE
Purpose	Run oversight, signals, approvals, command execution	Files, editor, terminal, agent dock
Center	Mission control — run phase, live feed, resume/complete	Monaco editor + bottom terminal dock
Left sidebar	Workspaces + **Attention** (signals, inbox, receipts)	Explorer / search / git activity bar
Right dock	Conversation transcript + Command/KAIRO hero	Resizable agent dock (conversation + composer)
Best for	"What is running? What needs me? Run this command."	"Edit files, use terminal, review code."
Input style	**Exact commands** in the Command seam (see footer **Commands**)	Same command seam in agent dock + full editor/terminal

Operator mode is the default production surface for day-to-day oversight.

IDE mode is for hands-on work in the bound repo (real `project_root` on disk).

Switch modes anytime — workspace selection, runs, and conversation thread persist.

What you can do in Axon-X today (v1)

Real and verified today:

- Select **axon-watch** or **axon-local** workspace
- View **runtime summary**, **inbox signals**, and **KAIRO briefing** from live APIs
- Track **run phase** in mission control (stop/resume/review-ready flows)
- Send **supported commands** (not free-form chat) via the Command seam
- Run **git status** against the bound repo root
- **IDE mode**: open workspace files in Monaco, PTY terminal in repo root
- **Attention sidebar**: connector/signal/delivery visibility

Still thin or deferred (use axon-local `:7734` fallback if needed):

- General conversational chat ("Hi", "explain this repo")
- Full agent tool loop parity with classic Axon
- Child-project connectors and legacy integration surfaces
- Native tray notifications beyond hide-on-close packaging

Supported commands (Operator mode)

The conversation/command seam accepts **exact commands** only. Natural language will return “unsupported command”.

COMMAND	WHAT IT DOES
`health` / `api/health`	Probe control-plane health
`ls` / `list files`	List files in the workspace
`read README.md` / `cat notes.txt`	Read a workspace file
`git status`	Git status in the bound project root
`resume from review`	Resume the primary `review_ready` run

In the UI: footer **Commands** button (Operator mode) opens the list and can prefill the Command seam. Source of truth: `apps/console-web/src/lib/operator-supported-commands.ts` (keep in sync with `services/control-plane/app/chat/command_executor.py`).

Typical first session

1. Open `:4173` → wait for boot overlay → shell loads
2. Left sidebar → select **axon-watch**
3. Operator mode → right dock → **Command** tab
4. Type `git status` → send → read agent receipt in **Conversation**
5. Center mission control shows run phase if a run was created
6. Toggle **IDE** → open `README.md`, use terminal in the real repo root
7. Left **Attention** → inspect signals when watch reports degraded summary

When things look noisy

Dev SQLite may contain old smoke runs (“32 runs ready for review”). Reset if needed:

```
./scripts/dev/down.sh
rm -f .local/state/control-plane.sqlite3
./scripts/dev/up.sh
```

Verify smoke

```
npm run verify:production-operator
```

Understanding runs, review_ready, RESUME, and COMPLETE

Analogy: Axon-X is a **supervised assistant**, not a chat bot. When you send a command like `read README.md` or `health`, the system starts a **run** (a tracked job): do the work → show output in **Conversation** → **pause** at a checkpoint called **review_ready**. That pause is intentional — you look at the result before anything else happens.

BUTTON / ACTION	PLAIN MEANING	WHEN TO USE IT
COMPLETE RUN	"I'm done — this job succeeded."	One-shot commands (`read …`, `git status`, `health`) when output looks good
RESUME RUN	"I've reviewed it — keep this run going."	Multi-step work, or when KAIRO **ADVISE** says to resume a named run
Command → `resume from review`	Same as RESUME for the **primary** paused run	When buttons are unclear; acts on one run at a time
Left → Attention	Signal inbox (bootstrap noise, delivery receipts)	When footer shows **SIGNALS: N ACTIVE** — usually informational in dev

"Phase is now review_ready. Review when ready." — not an error. The command finished; Mission Control is asking you to **COMPLETE** (usual) or **RESUME** (if more steps expected).

"2 runs are ready for operator review" — what that means

This is a **count of paused jobs**, not a failure. Each command you ran (`health`, `read README.md`, `git status`, …) can leave its own run sitting in **review_ready** until you **COMPLETE** or **RESUME** it. **2 runs** = two unfinished checkpoints (e.g. one Health run + one Read README run).

What to do:

1. **Center → Mission Control** — handles the **primary** (most recent) run via **RESUME RUN** / **COMPLETE RUN**.
2. **Right → KAIRO Briefing** — see **NOTICE** (the count) and **ADVISE** (suggested next click).
3. **Clear the backlog** — for each run you're happy with, click **COMPLETE RUN**. Repeat until the notice says "No active runs" or only one remains.
4. **Dev reset** (optional) — wipe stale runs from smoke tests:

```
./scripts/dev/down.sh && rm -f .local/state/control-plane.sqlite3 && ./scripts/dev/up.sh
```

KAIRO NOTICE / ADVISE / DECIDE — where to go

KAIRO Briefing (right dock → **KAIRO** tab, or footer **Open KAIRO Briefing**) is a **short summary**, not a second app. It mirrors the same backend truth as Mission Control and Attention.

LABEL	MEANING	WHERE TO ACT
NOTICE	Headline (“2 runs are ready…”)	Read only — context
ADVISE	Suggested next step (e.g. “Resume Health.”)	Usually → **RESUME RUN** in center, or **COMPLETE RUN** if that job is done
DECIDE	What choice is waiting	Center buttons, or Attention for signals
EXECUTE	Concrete action phrase	Command tab, or the button ADVISE points to

Example: ADVISE says **“Resume Health.”** → you previously ran `health` and that run is paused → go to **Mission Control** → **RESUME RUN** (continues that run) or **COMPLETE RUN** (closes it if you only wanted a one-time health check).

Signals (e.g. “Bootstrap: runtime summary stale”) in **Attention** are often **expected in local dev** — watch/bootstrap scaffolding, not production outage. Review them in **Left → Attention**; they do not block **COMPLETE RUN** unless an approval gate is open.

Bootstrap & signals — what to do

Bootstrap in Axon-X means the console and services are up, but some runtime summary fields are still intentionally thin while watch/control-plane parity grows. That is normal in local dev — not the same as “the app is broken.”

WHAT YOU SEE	WHAT IT MEANS	WHAT TO DO
Bootstrap: runtime summary stale (Attention → Signals)	Watch is connected; summary assembly is bootstrap-thin	**Ignore**, tap **Details**, or hit **CLEAR** to acknowledge and hide bootstrap noise
OBSERVE chip on a signal	KAIRO watch mode: informational only	**No click action.** Read-only label.
DELIVERED chip	Delivery receipt was recorded	**No click action.** Read-only label.
SIGNALS: N ACTIVE (footer)	N open inbox signals	Open **Attention** or **Open Attention** from Mission Control — review, then return to Command
IDLE + bootstrap signal only	Nothing waiting on you	No run action required — optional signal review only

When bootstrap noise is OK: local `./scripts/dev/up.sh`, watch healthy, no pending approvals, runs complete normally.

When to investigate: approvals stuck open, runs won’t resume/complete, watch disconnected in footer, or bootstrap signal persists **after** a production deploy with full runtime summary expected.

See also Tip 6: bootstrap-real vs feature-real.

Run names — what you see in the UI

Every command creates a **run** with two identifiers:

WHAT YOU SEE	MEANING
Health check, **Read README.md**, **Git status**	Friendly **task name** (from your command)
#cdb931 (6 characters)	Short **run ref** — internal tracking only; you rarely need the full `run_…` id

Old runs stored raw command text (`health` , `git status`); the UI **humanizes** those labels. KAIRO **ADVISE** uses the same friendly names (e.g. “Resume Health check.” not “Resume health.”).

When **2+ paused tasks** appear, Mission Control lists each friendly name — click **COMPLETE RUN** for the current one, repeat until the queue clears.

04 Runtime auth, CLI, and tools

This section explains how Axon-X authenticates agent runtimes, when you need secrets in **/vault**, and when Axon uses its own executors vs external CLIs.

Official Cursor CLI reference: Cursor CLI authentication

Two auth paths for Cursor (Pro / daily use vs headless)

PATH	WHEN TO USE	WHAT YOU DO	VAULT KEY REQUIRED?
CLI subscription (recommended)	Daily operator work on your machine with Cursor Pro/Team	Run `cursor agent login` once on the **host**; verify with `cursor agent status`	**No** — `CURSOR_API_KEY` is optional
API key (headless / CI)	Servers without browser, automation, cloud agents	Generate key in **Cursor Dashboard → Integrations → API Keys**; store as `CURSOR_API_KEY` in /vault or shell env	**Yes** (or shell env)

Axon-X probes subscription auth live:

- **Vault consumer** (`/vault` → Consumer readiness): marks **Cursor CLI runtime Ready** when `cursor agent status` shows a logged-in account, even with zero vault keys.
- **Control plane** (`GET /api/runtime/status` , `GET /api/runtime/cursor/status`): same probe; composer shows account + auth method.
- **Dispatch**: if subscription is active, control-plane **strips** `CURSOR_API_KEY` from the subprocess env to avoid auth conflicts (browser login and API key are alternate paths per Cursor docs).

Pro without vault key: if `cursor agent status` prints `Logged in as …@…` and vault search shows no `CURSOR_API_KEY` , you are on the **subscription path** — correct for daily Pro use.

Codex / OpenAI auth

PATH	SETUP
Codex CLI login	`codex login` on the host; vault consumer probes `codex login status`
API keys in vault	`CODEX_API_KEY` or `OPENAI_API_KEY` in /vault (either satisfies the codex consumer)

Vault consumers vs runtime dispatch

Consumers are readiness labels for operators — they never expose secret values.

CONSUMER	READY WHEN	OPTIONAL / FALLBACK
`cursor_runtime`	CLI subscription **or** `CURSOR_API_KEY` in vault	API key only needed for headless
`codex_runtime`	`codex login` **or** Codex/OpenAI vault keys	
`openai_provider`	`OPENAI_API_KEY` in vault	Direct OpenAI fallback
DashPro monitor consumers	Required monitor keys in vault/import	

Source: `services/axon-watch/app/vault/snapshot.py` , `cli_runtime_probe.py`

Runtime dispatch (actually running a model) lives in the control-plane:

- `services/control-plane/app/cli_runtime/catalog.py` — auth probes, ready flags
- `services/control-plane/app/cli_runtime/router.py` — picks binary, env, retry without API key on oauth
- `services/control-plane/app/cli_runtime/vault_keys.py` — merges unlocked vault keys into runtime context

Native Axon tools vs Cursor CLI

Axon-X has **two execution lanes** — do not mix them when debugging.

SURFACE	LANE	EXECUTOR	TOOLS
Operator mode → Command seam	**Lane A — Command**	Axon `command_executor.py`	Exact commands only (`health`, `git status`, `read …`) — **Axon native**
IDE mode → Agent dock composer	**Lane B — Ask / Plan / Agent**	Cursor CLI subprocess (`cursor agent …`)	Cursor model + consultative prompt; **not** full Cursor IDE agent tools yet
Future (G3.5+)	MCP registry	Planned wiring	Static registry exists; **not** connected to dispatch today

When to use native Axon tools: Operator oversight, deterministic repo commands, health checks, file reads — anything in the supported commands table.

When Cursor CLI runs: IDE composer messages in Ask, Plan, or Agent mode with runtime target **Cursor CLI (local)**. Streaming (SSE) applies to this lane when `AXON_WATCH_LANE_B_STREAMING=1` (default).

Composer modes (IDE dock):

MODE	CURSOR CLI FLAG	BEHAVIOR TODAY
Ask	`--mode ask`	Read-only style answers
Plan	`--mode plan`	Step mapping before execution
Agent	Default consultative; **Full Access** in composer → approval → Cursor `--mode agent` / Codex workspace-write	`execution_access: full` + G3.3 approval gate

Choosing runtime target and model

Open the **model picker** (⚡ chip) in the IDE agent dock:

1. **Runtime target** — `Cursor CLI (local)`, `Codex CLI (local)`, or cloud placeholders. Preference persists in shell local storage via `shell.setSelectedRuntimeTarget`.
2. **Auto toggle** — when ON, Cursor picks the best model per request (no `--model` flag). When OFF, your pinned catalog model is passed to the CLI.
3. **Add models** — browse live output of `cursor agent --list-models` (cached ~5 min). Badges like **Fast** / **High** come from CLI labels when present.
4. **Auth line** — shows `CLI subscription · you@domain` or vault/API-key status; **Open Vault** only when auth is actually blocked (vault locked or missing keys **and** CLI not signed in).

Environment overrides:

VARIABLE	PURPOSE
`AXON_WATCH_CURSOR_CLI_PATH`	Non-default `cursor` binary path
`AXON_WATCH_CODEX_CLI_PATH`	Non-default `codex` binary path
`AXON_WATCH_LANE_B_STREAMING`	`1` (default) SSE streaming for IDE composer; `0` in tests

API endpoints:

- `GET /api/runtime/status` — all targets + vault posture
- `GET /api/runtime/cursor/status?force_refresh=1` — Cursor auth + live model catalog
- `GET /api/vault/status` — consumer readiness (axon-watch service)

Troubleshooting auth

SYMPTOM	LIKELY CAUSE	FIX
Vault shows **Missing: CURSOR_API_KEY** but CLI is logged in	Stale UI or old snapshot	Refresh /vault; consumer should show **CLI subscription · …**
Composer says invalid API key	Stale `CURSOR_API_KEY` in vault **or** shell env conflicting with subscription	Remove bad key from vault; unset shell `CURSOR_API_KEY`; restart control-plane
`cursor agent status` → not logged in	No subscription session on host	`cursor agent login`
Runtime ready but dispatch fails	Wrong binary or model id	Check `AXON_WATCH_CURSOR_CLI_PATH`; pick **Auto** or a model from live catalog
Open Vault when Pro works	Should not show if oauth ready	Hard-refresh; verify `GET /api/runtime/status` shows `ready: true`
Out of usage / rate limit from Cursor	Current CLI account hit subscription quota	Switch accounts (below) or use **Auto** / another model; admin may need to raise team limits

Switch Cursor subscription account (logout → login)

When the agent reports **out of usage** or you need a different Pro/Team account on this machine:

```
# 1. Sign out (clears stored CLI session on this host)
cursor agent logout

# 2. Confirm logged out
cursor agent status   # should say not logged in / auth required

# 3. Sign in with the other Cursor account (opens browser)
cursor agent login

# 4. Verify the new account
cursor agent status   # expect: Logged in as other@domain
```

Headless terminal (no browser auto-open):

```
NO_OPEN_BROWSER=1 cursor agent login
# Open the printed URL manually in a browser logged into the target account
```

After switching:

1. Refresh **/vault** → Cursor consumer should show the new **CLI subscription · email**
2. Hard-refresh `:4173` or reopen the model picker (forces runtime status refresh)
3. Retry IDE composer — old thread errors are from the prior account/session; start a new turn

Note: Axon-X does not store Cursor passwords. Logout/login only affects the **host Cursor CLI** session. Vault `CURSOR_API_KEY` (if present) is separate — remove or update it if you switch to API-key auth instead.

Quick host checks:

```
cursor agent status          # expect: Logged in as …
echo "${CURSOR_API_KEY:+set}" # empty = good for Pro path
curl -s http://127.0.0.1:8787/api/runtime/cursor/status | python3 -m json.tool
```

05 VAXON Desktop

See `docs/how-to/vaxon-desktop.md`.

06 Terminology And Abbreviations

Use this glossary when reading plans, ADRs, code, or agent summaries.

TERM	MEANING
ADR	Architecture Decision Record — a numbered, immutable write-up of a significant technical or process choice. Accepted ADRs live in `docs/adr/`. Do not rewrite them; supersede with a new ADR instead.
Axon-X	User-facing product name for the next-generation operator console.
axon-watch	Internal repo folder and npm workspace name. Legacy naming; not the product label shown to operators.
axon-local	The current production Axon app repo (port **7734**). Planning for Axon-X still lives here under `Plans/Axon-Watch/`.
Bootstrap	Early boot / dev scaffolding state — services run, but some DTO fields (runtime summary depth, signal richness) are intentionally thin until parity slices land. Signals like `Bootstrap: runtime summary stale` are **expected locally**, not production outages.
Briefing seam	`GET /api/briefing` returns canonical `OperatorBriefing`. The shell loads it at bootstrap and projects that data across the right dock: approvals, signals, and the KAIRO briefing card all read from the same briefing/runtime truth. Approval mutations stay on the run approval seam. See `docs/contracts/BRIEFING-SEAM.md`.
Control plane	FastAPI service on port **8787** that owns run truth, runtime summary, inbox projection, workspaces list, and briefing.
Console-web / shell	Vue 3 frontend on port **4173** — the visible Axon-X UI.
Contract / shared contract	Canonical TypeScript types and JSON fixtures in `packages/shared-types/`. Frontend and backend must agree here first.
Coordinator lane	Single owner of serial-only semantics during multi-agent work. See `docs/MULTITASK-LANES.md`.
DTO	Data Transfer Object — a typed payload shape exchanged between services or UI layers (for example `RuntimeSummary`, `RunRecord`).
Fitness function	An automated check that guards architecture or performance (dependency direction, DTO size budgets, latency thresholds).
Frozen planning bundle	The locked docs under `axon-local/Plans/Axon-Watch/`. Implementation must not silently drift from these.
KAIRO	Knowledge-Augmented Intelligence for Response and Oversight — operator-presence layer (watching, advising, interrupting, executing with receipts). **NOTICE / ADVISE / DECIDE** in the KAIRO Briefing panel are rhythm labels from `GET /api/briefing` — suggested reading, not separate commands. See Quick Start → *KAIRO NOTICE / ADVISE*.

TERM	MEANING
Lane A/B/C/D	Parallel implementation ownership areas defined in `docs/MULTITASK-LANES.md` (watch, shell, control-plane, dev/verify).
Monaco host	In-browser code editor surface (`EditorHost.vue`). Loads workspace files on disk (README.md, notes.txt) plus read-only DTO overview tabs.
Parity ledger	Checklist of behaviors Axon-X must eventually match from current Axon. Lives in frozen planning.
Run / run record	Canonical execution unit with `phase` , `status` , capability flags, and transition history.
Run phase	High-level lifecycle stage (`queued` , `starting` , `executing` , `awaiting_approval` , `review_ready` , `completed` , etc.). Defined in frozen `run-state.md` .
Runtime summary	Boot-critical DTO at `GET /api/runtime/summary` — identity, watch connection, active runs, approvals snapshot.
Signal / inbox item	Watch-produced event surfaced through control-plane `GET /api/inbox` .
Thin slice	A small, verifiable vertical increment — one owned behavior with tests, not a broad rewrite.
Watch service	FastAPI service on port **8788** that produces canonical signals; control-plane projects them into inbox/runtime summary.
xterm host	In-browser terminal surface (`TerminalHost.vue`) attached to a **backend PTY session** via `WS /api/workspaces/{workspace_id}/terminal` . Runs real shell commands in a workspace-scoped directory under `.local/workspaces/` (override with `AXON_WATCH_WORKSPACE_ROOT`).
Workspace	Logical operator context keyed by `workspace_id` . Today the API returns IDs only; rich catalog metadata is deferred.

Two repos, two apps

Do not confuse these:

	AXON-LOCAL (CURRENT AXON)	AXON-WATCH (AXON-X)
Default URL	`http://127.0.0.1:7734` (fallback)	`http://127.0.0.1:4173` (**production operator**)
Start command	`./start.sh` from axon-local	`./scripts/dev/up.sh` from axon-watch
Status	Legacy daily-driver / fallback	Primary operator console (v1)
Relationship	Source of parity targets and frozen plans	Implementation target for modernization

07 What Axon-X Is

Axon-X is the next-generation Axon console and operator environment.

The code lives in the `axon-watch` repo folder for now. That folder name is legacy/internal; the product name is **Axon-X**.

The target product combines:

- an IDE-style shell
- a control plane
- a dedicated watcher service

The implementation repo is here:

- `/home/edp/axon-nvme/repos/axon-watch`

The frozen planning source-of-truth still lives here:

- `/home/edp/axon-nvme/repos/axon-local/Plans/Axon-Watch/`

Important rule:

- read plans from `axon-local`
- implement in `axon-watch`

Do not casually move back and forth inventing new semantics in both places.

08 What State The Repo Is In Right Now

The repo is in an early but real bootstrap state.

What already exists:

- repo scaffold
- `console-web` shell skeleton
- `control-plane` FastAPI bootstrap service
- `axon-watch` FastAPI bootstrap service
- shared contract package
- verification harness
- health and readiness scripts

What is already real:

- root workspace scripts
- service startup/shutdown flow
- `/api/runtime/summary`, `/api/inbox`, `/api/runs`, and `/api/briefing` routes in the control plane
- shared contract types under `packages/shared-types/`
- shell consumption of canonical runtime summary, inbox, and run DTOs
- thin Monaco and xterm host surfaces in `console-web`

What is still intentionally thin or deferred:

- full signal production and deeper ranking beyond current inbox rule stack
- deep watch summary logic
- performance evidence for all budgets

What is now real in the thin slice (verified 2026-07-04):

- run create/complete/stop/resume lifecycle
- explicit approval boundary (`requires_approval`, approve/reject)
- review-ready entry, completion, and follow-up resume path
- SQLite-backed run persistence (survives control-plane restart)
- operator briefing loaded at bootstrap and rendered in the right dock (`BriefingPanel.vue`)
- two watch-produced inbox signals with multi-factor ranking (severity, recency, unresolved duration via `created_at`, status, action-type, workspace priority)
- workspace list API and shell workspace selector (IDs only)

- the shell is split into `TopBar`, `LeftSidebar`, `CenterWorkbench`, `RightDock`, and `StatusBar` regions with mockup-shell chrome
- Monaco host bound to canonical DTO documents and **workspace files on disk** with a nested explorer tree, lazy file loading, new-file creation, and active-file rename
- backend PTY terminal attachment for the selected workspace (real shell I/O via WebSocket)
- workspace-scoped conversation rehydration: `GET /api/workspaces/{workspace_id}/chat/thread` plus existing thread history read reloads the Conversation seam after page refresh. When no thread exists yet, the lookup returns HTTP 200 with null `thread_id` (not 404).

Workspace IDs (operator vs catalog):

- **Operator shell** uses `MOCKUP_WORKSPACE_IDS` only (`workspace_smoke`, `workspace_recsys`, …). `mergeMockupWorkspaceCatalog()` trims API extras so `currentWorkspace` is always sidebar-visible.
- **Control-plane catalog** may still list fixture defaults (`workspace_alpha`, `workspace_bootstrap`) and run/inbox IDs for tests and API consumers; the shell does not select those as `currentWorkspace`.
- Bootstrap picks workspace deterministically: active run workspace (when visible) → `workspace_smoke` default → first mockup workspace.

Manual acceptance for reload-safe chat (use **`workspace_smoke`**):

1. `./scripts/dev/up.sh`
2. Open `http://127.0.0.1:4173`
3. Confirm **`workspace_smoke`** is selected (or select it)
4. Post a command in the Command seam
5. Hard reload the page
6. Conversation should rehydrate automatically for the same workspace when an active run or default bootstrap applies; if needed, re-select **`workspace_smoke`**

API-only check (any valid catalog ID):

```
curl -s http://127.0.0.1:8787/api/workspaces/workspace_smoke/chat/thread
curl -s http://127.0.0.1:8787/api/chat/threads/<thread_id>/history
```

What is **not** real yet despite similar-sounding names:

- **Full KAIRO operator presence** — the current shell has visual scaffolding, but spoken alerts, persona settings, and richer operator-presence behavior are still planned in axon-local docs
- **Full parity with axon-local** — intentional; see parity ledger for gaps

So this repo is not a fake mockup, but it is also not feature-complete or a drop-in replacement for the current Axon UI.

09 Source Of Truth Rules

When you are unsure what something should mean, check the planning bundle.

The most important frozen planning docs are:

- `PRODUCT.md`
- `ARCHITECTURE.md`
- `UI_SPEC.md`
- `UI_COMPOSITION_SPEC.md`
- `UI_VISUAL_DIRECTION.md`
- `UI_REFERENCE_ARCHETYPES.md`
- `run-state.md`
- `runtime-summary.md`
- `signal-events.md`
- `watch-api.md`
- `control-api.md`
- `PARITY_LEDGER.md`
- `FITNESS_FUNCTIONS.md`
- `TRANSITION_ARCHITECTURE.md`
- `CONTRACT_TESTING_SPEC.md`

Things you must not redefine casually

These are serial-only semantics and should not drift:

- canonical run phases
- runtime summary vocabulary
- signal identity, severity, and status
- approval semantics
- transition seams and rollback rules
- parity acceptance rules

If the plan seems wrong or insufficient:

- do not silently change it
- raise a proposed amendment instead

10 Repo Layout

Top-level structure:

```
axon-watch/
  apps/
    console-web/
  services/
    control-plane/
    axon-watch/
  packages/
    shared-types/
  docs/
  scripts/
    dev/
    verify/
  infra/
  tests/
```

What each part owns

`apps/console-web/`

Owns the integrated shell:

- topbar
- left sidebar
- center workbench (including the embedded terminal dock)
- right dock
- status bar

It should consume canonical contracts, not invent backend truth.

`services/control-plane/`

Owns interactive backend responsibilities such as:

- UI-facing health/readiness
- runtime summary assembly
- later run-state, approvals, and workspace orchestration

`services/axon-watch/`

Owns watcher responsibilities such as:

- monitoring loops
- watch health/readiness

- later signal production
- later inbox/signal summaries

`packages/shared-types/`

Owns canonical shared contract types.

This is one of the most important directories in the repo.

If frontend and backend disagree about what a thing means, the fix should start here, not in ad hoc local types.

`scripts/dev/`

Owns local run helpers:

- start
- stop
- health checks

`scripts/verify/`

Owns cheap, repeatable verification:

- dependency direction checks
- DTO size checks
- ADR governance checks
- latency-budget check scaffolding

11 The Current Architecture In Plain English

Think of the new app as three cooperating parts:

1. the shell the user sees
2. the control plane that serves user-facing APIs
3. the watcher service that will eventually observe and normalize signals

The current implementation is following a thin-slice path:

- bootstrap the shell and services first
- land canonical contracts early
- add real endpoints against those contracts
- then deepen behavior

This is deliberate.

It avoids:

- giant rewrites
- hidden semantic drift
- UI and backend inventing different meanings

12 Detailed setup (first install)

1. Go to the repo

```
cd /home/edp/axon-nvme/repos/axon-watch
```

2. Install JavaScript dependencies

```
npm install
```

This is important because root workspace commands now rely on npm workspaces.

3. Review environment defaults

Default values are provided in:

- `.env.example`

If you need custom ports or local paths:

```
cp .env.example .env
```

Then edit `.env` as needed.

4. Start the local stack

```
./scripts/dev/up.sh
```

`up.sh` does not report success until all three services are actually reachable.

This starts:

- `console-web`
- `control-plane`
- `axon-watch`

Important startup contract:

- ports are fixed by `.env` / `.env.example`
- startup fails fast if `4173`, `8787`, or `8788` is already in use
- startup rolls back partial processes if readiness fails
- services stay detached after the launching shell exits

5. Check health

```
./scripts/dev/check-health.sh
```

Expected endpoints:

- console web: `http://127.0.0.1:4173`
- control plane health: `http://127.0.0.1:8787/api/health`
- watch health: `http://127.0.0.1:8788/internal/watch/health`
- briefing: `http://127.0.0.1:8787/api/briefing` (also loaded by the shell at bootstrap)
- runtime summary: `http://127.0.0.1:8787/api/runtime/summary`
- inbox: `http://127.0.0.1:8787/api/inbox`
- runs: `http://127.0.0.1:8787/api/runs`
- workspaces: `http://127.0.0.1:8787/api/workspaces`

`check-health.sh` also probes `/api/workspaces`.

6. Stop the stack

```
./scripts/dev/down.sh
```

13 What The Current App Does On Boot

Right now, the shell boots and loads five control-plane seams in parallel.

That flow looks like this:

1. `apps/console-web/src/main.ts` creates the Vue app
2. the shell store initializes
3. `loadBootstrapData()` is called
4. the frontend fetches `/api/runtime/summary`, `/api/inbox`, and `/api/briefing`
5. workspaces and runs load sequentially; `resolveBootstrapWorkspaceId()` sets `currentWorkspace` (active run workspace when sidebar-visible, else `workspace_smoke`)
6. workspace files and chat thread history load for that workspace
7. the shell renders topbar context, workspace state, editor/terminal workbench, right-dock seams, and status bar truth strips

Important limitations:

- **Operator workspace list** — shell and sidebar share the same `MOCKUP_WORKSPACE_IDS` catalog; catalog-only IDs such as `workspace_alpha` remain API-visible but are not selected in the shell
- **Bootstrap workspace selection** — deterministic via `resolveBootstrapWorkspaceId()` after sequential workspace + run load (no parallel race)
- **Chat rehydration** — scoped per workspace; messages posted under one ID do not appear when another workspace is selected; only the latest thread per workspace is returned
- **Chat orchestration** — `POST /api/chat/messages` returns operator + system + agent messages; new dispatches run bounded executor (`health` / `list files` / `read …`) then `review_ready` with `command_execution` receipt
- **Workspace catalog** — sidebar uses seven mockup IDs; API may expose more — see `docs/WORKSPACE_CATALOG.md`
- Operator mode renders the right dock as **Run → Approvals → Signals → Conversation → Command → KAIRO Briefing** with the briefing card anchored at the bottom of the dock
- briefing data is projected across the dock rather than shown as one raw DTO dump: approvals stay in the approvals seam, top signals stay in the signals seam, and the KAIRO card stays summary/CTA-oriented
- Monaco host loads workspace files on disk and read-only DTO overview tabs
- xterm host attaches to a backend PTY scoped to the selected workspace directory
- inbox ranking uses severity, recency, unresolved duration (`created_at`), status, action-type, and a thin watch-owned workspace priority map

That is okay for this stage.

14 Locked Shell Layout

Locked 2026-07-04 — see `docs/UI_LAYOUT_LOCK.md` and `docs/adr/ADR-004-locked-console-shell-layout.md`.

Do not rearrange shell regions or dock seam order without a superseding ADR. Frozen planning in `axon-local/Plans/Axon-Watch/UI_COMPOSITION_SPEC.md` is amended to match this geometry.

Five-region grid:

REGION	COMPONENT	NOTES
Top bar	`TopBar.vue`	identity zone + runtime strip + KAIRO module + mode toggle
Left sidebar	`LeftSidebar.vue`	workspaces, optional explorer (IDE), status card
Center workbench	`CenterWorkbench.vue`	Monaco editor + embedded resizable terminal dock
Right dock	`RightDock.vue`	Run → Approvals → Signals (upper stack), KAIRO Briefing bottom hero
Status bar	`StatusBar.vue`	HUD runtime strip

KAIRO Briefing height tracks the workbench terminal dock via `--briefing-dock-height`.

15 Current Shell Layout

The locked shell matches the mockup/live screenshots:

- `TopBar` — brand frame, mockup breadcrumb/version context panel, DTO runtime strip, KAIRO presence module, Operator/IDE toggle, settings action
- `LeftSidebar` — workspace list first, workspace status card second, explorer tree only in IDE mode
- `CenterWorkbench` — editor tabbar + breadcrumb + Monaco editor above a resizable bottom terminal/log dock
- `RightDock` — run seam, approvals seam, signals seam, KAIRO briefing card
- `StatusBar` — persistent watch / run phase / signals / operator strip

This is the layout you should compare against the mockup and live screenshots, not the older single-file shell description from earlier thin slices.

16 Mockup / Live Parity Notes

Parity observations from the current mockup and live screenshots:

- the region geometry now largely matches the mockup: topbar, workspace rail, editor-over-terminal workbench, right dock, and bottom status strip
- the **runtime strip**, status bar zones, and workspace status card derive from live shell state / `RuntimeSummary`, but the topbar breadcrumb and runtime version chips are still mockup-style presentation helpers
- the sidebar and shell store share the same operator workspace catalog (`MOCKUP_WORKSPACE_IDS`)
- the dock uses operator-facing seam titles (`Active Run` , `Approvals` , `Signals` , `Conversation`) from `dock-seam-layout.ts`
- live shell refresh uses `GET /api/live/events` (SSE refresh hints) via `live-events-session.ts` , with visibility-aware polling fallback when EventSource is unavailable
- Operator mode renders the right dock as **Run → Approvals → Signals → Conversation → Command → KAIRO Briefing** with the briefing card anchored at the bottom of the dock
- the workbench terminal dock default height is ~240px (responsive cap 280px) unless the operator resizes it; height persists in session storage when customized

17 The Most Important Files Right Now

If you need to understand the current implementation quickly, read these first:

Shared contracts

- `packages/shared-types/src/index.ts`
- `packages/shared-types/src/run.ts`
- `packages/shared-types/src/runtime.ts`
- `packages/shared-types/src/signals.ts`
- `packages/shared-types/src/control-plane.ts`
- `packages/shared-types/src/watch.ts`

Fixture payloads

- `packages/shared-types/fixtures/runtime-summary.example.json`
- `packages/shared-types/fixtures/watch-summary.example.json`
- `packages/shared-types/fixtures/run-record.example.json`
- `packages/shared-types/fixtures/signal-event.example.json`

Control plane

- `services/control-plane/app/main.py`
- `services/control-plane/app/workspace_catalog.py`
- `services/control-plane/app/runs/service.py`
- `services/control-plane/app/runtime_summary_assembler.py`
- `services/control-plane/app/operator_briefing.py`

Frontend shell

- `apps/console-web/src/main.ts`
- `apps/console-web/src/api/control-plane.ts`
- `apps/console-web/src/stores/shell.ts`
- `apps/console-web/src/App.vue`
- `apps/console-web/src/components/EditorHost.vue`
- `apps/console-web/src/components/TerminalHost.vue`
- `apps/console-web/src/lib/workspace-documents.ts`

Verification

- `scripts/verify/README.md`
- `scripts/verify/verification_config.json`
- `tests/test_shared_contract_fixtures.py`
- `tests/test_control_plane_runtime_summary.py`
- `tests/test_verify_harness.py`

18 Verification Commands

Use these from the repo root.

CI, merge to `dev`, and employee agents: see docs/how-to/ci-merge-and-worker-agents.md (Fast Gate workflow, PR workflow, roster/scheduler, how to tell if workers are live).

Shared contract verification

```
npm run verify:shared-types
```

This confirms the shared-types package typechecks.

Contract verification

```
npm run verify:contracts
```

This verifies:

- shared contract package typing
- shared fixture tests
- control-plane run/inbox/runtime-summary/briefing behavior
- watch signal contract alignment

Full current verification bundle

```
npm run verify
```

This runs:

- contract verification
- console-web typecheck, unit test, and production build
- verify harness checks
- DTO size checks using representative fixtures

TEST-0 acceptance (`workspace_smoke`)

Requires the dev stack (`./scripts/dev/up.sh`).

```
npm run verify:test0
```

This runs, in order:

1. `./scripts/dev/check-health.sh`

2. Mission control unit tests (`operator-status-radar-view` , `workbench-terminal-split`)
3. Live acceptance — `tests/test_test0_workspace_smoke_acceptance.py` against `workspace_smoke` (briefing Notice/Advise, git status, resume from review, inbox/runs)
4. `npm run verify`

See also `docs/OPERATOR_MISSION_CONTROL_V1.md` for the manual UI checklist.

20 Latency evidence (D1)

When the dev stack is running, collect warm-route timing samples and re-run verify with evidence files:

```
./scripts/dev/up.sh
npm run verify:evidence
npm run verify:nightly
```

This writes JSON under `.local/verify/` and passes them to `scripts/verify/all.py` .

Shell boot measurement:

- `scripts/dev/measure_shell_boot.py` records `shell_ready_ms`
- `auto` mode uses Playwright Chromium when installed; otherwise it measures the bootstrap critical path (index fetch + parallel `/api/runtime/summary` , `/api/inbox` , `/api/briefing` , `/api/workspaces` , `/api/runs`)
- optional full browser mode: `pip install playwright && playwright install chromium` then `AXON_WATCH_SHELL_BOOT_MODE=browser npm run verify:evidence`
- repo-local setup:

```
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements-dev.txt
python -m playwright install chromium
```

`collect-verify-evidence.sh` prefers `.venv/bin/python3` when present so `auto` mode can use Playwright.

Example fixture: `scripts/verify/fixtures/shell-boot-report.dev.json` .

`./scripts/dev/check-health.sh` also probes `GET /api/live/events` (SSE).

Frontend checks

```
npm run typecheck -w @axon-watch/console-web
npm run test -w @axon-watch/console-web
npm run build -w @axon-watch/console-web
```

Python syntax checks

```
python3 -m py_compile services/control-plane/app/main.py services/control-plane/app/runs/service.py services/control-plane/app/domain/run_state.py services/control-plane/app/domain/run_transitions.py services/control-plane/app/operator_briefing.py
```

Service tests

```
python3 -m unittest discover -s tests
```

21 What PASS / PENDING / FAIL Mean In Verification

The verify harness uses explicit states.

`PASS`

The supplied evidence met the rule.

Examples:

- DTO payload fits its size budget
- dependency direction rule is respected

`PENDING`

The check exists, but the slice does not provide enough real implementation or evidence yet.

Examples:

- shell boot timing evidence not supplied yet
- runtime summary latency budget not yet measured
- no numbered ADR yet exists

`PENDING` is not automatically bad at this stage.

It means the governance has been created ahead of the full implementation.

`FAIL`

The rule was violated or the harness itself is broken.

This is the one that should stop you.

22 Common Working Patterns

When adding frontend work

Use real shared types from `packages/shared-types/`.

Do not:

- invent local copies of DTOs
- widen semantics casually in component code
- hide backend meaning in placeholder strings

Good pattern:

- define or refine contract in shared-types
- use it in store/API layer
- render it in Vue components

When adding control-plane work

Prefer:

- canonical DTO assembly
- light endpoint handlers
- explicit payload validation

Avoid:

- stuffing frontend-specific assumptions into the backend
- returning huge boot payloads
- redefining contract shapes inline

When adding watch work

Prefer:

- narrow, owned summaries
- canonical signal/event identities
- explicit watch responsibilities

Avoid:

- pulling UI logic into the watch service
- importing control-plane domain semantics directly
- inventing competing signal vocabulary

23 Troubleshooting

24 Debugging playbook

Use this order when something breaks:

1. **Stack health** — `axonhealth` (or `./scripts/dev/check-health.sh`).
2. **Empty shell / Runtime unavailable** — `axonrevive`, then hard-refresh `:4173`. Do **not** rely on `./scripts/dev/down.sh` when systemd owns the ports.
3. **Soft refresh** — `axonrestart` after backend route changes (if the API still answers).
4. **Stale UI on `:4173`** — rebuild console-web (`npm run build -w @axon-watch/console-web`), `systemctl --user restart console-web.service`, then hard-refresh. Source-only / `:5173` Vite edits do **not** update the systemd bundle.
5. **Browser cache** — hard refresh `:4173` (`Ctrl+Shift+R`) after console-web bundle changes.
6. **Connectors truth** — Mission Control → **Connectors** rail or `GET /api/connectors`.
7. **Gate scripts** — `npm run verify:production-operator`, then the slice gate (`verify:testN` / `verify:shell-commands`).
8. **Logs** — `journalctl --user -u control-plane.service -n 80` (always-on) or `.local/logs/` (dev bootstrap).

Symptom-specific fixes continue in the sections below.

Problem: Runtime unavailable / No workspace selected / Briefing unavailable

This is the **wedged control-plane** pattern. The Vue shell is up, but `/api/*` times out (even `/api/health`). Bootstrap never selects a workspace, so explorer + chat look empty.

```
axonrevive
# hard-refresh http://127.0.0.1:4173
```

Why `./scripts/dev/down.sh` / `up.sh` fail here: this host runs **user systemd units** (`control-plane.service`, etc.). Dev down/up skip those listeners. Soft `systemctl --user restart` can also hang on a stuck worker — `axonrevive` force-kills first.

Problem: online research falls back / Google search returns 403

See docs/how-to/searxng-research.md for SearXNG setup, provider order (SearXNG → legacy Google → DuckDuckGo), and Google 403 troubleshooting.

Problem: `./scripts/dev/up.sh` fails or the frontend does not start

Check:

1. Did you run `npm install` at repo root?
2. Are ports `4173`, `8787`, and `8788` free?
3. Did `up.sh` print a specific port conflict or readiness failure message?

Current expected path:

- root install: `npm install`
- root startup: `./scripts/dev/up.sh`
- health check: `./scripts/dev/check-health.sh`

If startup fails, inspect:

- `.local/logs/console-web.log`
- `.local/logs/control-plane.log`
- `.local/logs/axon-watch.log`

Problem: `check-health.sh` fails

This usually means one or more services never started.

Steps:

1. stop everything:

```
./scripts/dev/down.sh
```

2. start again:

```
./scripts/dev/up.sh
```

3. inspect logs:

- `.local/logs/control-plane.log`
- `.local/logs/axon-watch.log`
- `.local/logs/console-web.log`

4. re-run:

```
./scripts/dev/check-health.sh
```

Problem: stale pid files block startup

`up.sh` now clears stale pid files automatically before it checks for a live stack.

First try:

```
./scripts/dev/down.sh
```

If that is not enough, inspect:

- `.local/pids/`
- `.local/logs/console-web.log`
- `.local/logs/control-plane.log`
- `.local/logs/axon-watch.log`

If `up.sh` still fails, it usually means one of the configured ports is held by another live process. Free the port or stop the other process before retrying.

Problem: the shell loads but runtime summary is unavailable

Check:

1. Is control-plane running?
2. Does this endpoint work?

```
curl -fsS http://127.0.0.1:8787/api/runtime/summary | python3 -m json.tool
```

3. If running through Vite dev server, is `/api` proxied to control-plane?

Current dev proxy lives in:

- `apps/console-web/vite.config.ts`

You can also override the target with:

- `VITE_CONTROL_PLANE_BASE_URL`

Problem: `npm run verify` shows `PENDING`

That is often expected at this stage.

Examples of currently expected pending checks:

- shell boot readiness
- runtime summary latency
- watch summary latency
- numbered ADR presence

Treat `PENDING` as:

- “scaffold exists, evidence not landed yet”

Treat `FAIL` as:

- “something is wrong right now”

Problem: dependency direction checks fail

Check for forbidden imports across boundaries.

The current verify harness guards things like:

- watch service must not depend on UI internals
- frontend must not depend on watch internals
- future domain-layer cross-imports must stay clean

If a dependency check fails:

1. remove the cross-boundary import
2. move shared meaning into `packages/shared-types/` if appropriate
3. use an adapter or API boundary instead of direct reach-through

Problem: contract tests fail after changing fixtures or DTOs

This usually means one of three things:

1. you changed a canonical field name
2. you widened/narrowed a type without updating the fixture
3. you changed semantics without updating the contract layer first

Fix sequence:

1. check the frozen planning doc
2. check the shared contract type
3. check the fixture
4. check the consumer/provider test

Do not patch the frontend alone and ignore the canonical contract.

Problem: you are unsure whether something should become a new ADR

Ask:

1. Is this a real architecture or process decision?
2. Is it broader than one small implementation detail?
3. Would future readers need to know why this choice was made?

If yes, use ADR governance.

Current ADR process docs live in:

- `docs/adr/README.md`
- `docs/adr/_TEMPLATE.md`

Note:

- accepted ADRs should be numbered
- accepted ADRs should not be rewritten materially
- changed decisions should be superseded, not silently edited

25 Tips And Tricks

26 Snippet cookbook

One-word stack commands (always-on host)

Installed on PATH via `~/.local/bin` → repo `bin/`:

COMMAND	EXPANDS TO	WHEN TO USE
`axonhealth`	`./scripts/dev/check-health.sh`	Quick “is the stack OK?”
`axonrestart`	`systemctl --user restart` axon-watch + control-plane + console-web, then health	Soft restart when APIs still respond
`axonrevive`	Force-kill control-plane → restart all three → health	Empty shell, hung health, wedged worker

```
axonhealth
axonrestart
axonrevive
```

Repo scripts (same behavior without PATH):

```
./scripts/ops/axonhealth.sh
./scripts/ops/axonrestart.sh
./scripts/ops/axonrevive.sh
```

Open console after revive: **http://127.0.0.1:4173** (hard-refresh).

Console-web rebuild (always-on `:4173`)

`:4173` serves the **built** `apps/console-web/dist` via systemd `console-web.service`. Source edits (including Vite `:5173`) are **not** live on `:4173` until:

```
npm run build -w @axon-watch/console-web
systemctl --user restart console-web.service
# then hard-refresh http://127.0.0.1:4173
```

Local verify loop

```
./scripts/ops/change-verify-loop.sh             # dirty working tree
./scripts/ops/change-verify-loop.sh --head-only  # committed HEAD only
./scripts/ops/change-verify-loop.sh --watch
```

Dev bootstrap (when systemd is not owning ports)

```
./scripts/dev/up.sh
./scripts/dev/down.sh
./scripts/dev/check-health.sh
```

27 Upgrading and updating

After pulling new commits or changing dependencies:

```
cd /home/edp/axon-nvme/repos/axon-watch
git pull
npm install
./scripts/dev/down.sh && ./scripts/dev/up.sh
npm run verify:production-operator
```

Sync planning docs back to axon-local when Axon-Watch planning changed:

```
python3 scripts/ops/sync_planning_mirror_to_axon_local.py
```

Hard-refresh `:4173` after frontend changes. Restart the stack after control-plane or watch route changes.

Tip 1: Read the planning bundle before expanding semantics

The planning bundle is not optional context. It is the definition of intended meaning.

Tip 2: Treat `packages/shared-types/` as sacred

If the frontend and backend disagree, fix it here first.

Tip 3: Keep slices narrow

The repo is intentionally growing by thin slices.

If you are touching:

- run-state
- runtime summary
- signals
- approvals

then keep the slice tightly bounded and verifiable.

Tip 4: Prefer fixtures early

Fixtures are useful in early slices because they:

- keep contracts concrete
- make DTO size checks easy
- let frontend and backend agree before deeper logic exists

Tip 5: Use the verify harness even when it is not strict

`PENDING` today becomes `PASS` or `FAIL` tomorrow.

The harness is part of how this repo avoids drifting back into a monolith.

Tip 6: Distinguish “bootstrap-real” from “feature-real”

Something can be real enough to run and still be intentionally shallow.

Current examples:

- runtime summary endpoint is real
- runtime summary assembly is still bootstrap-thin
- axon-watch emits `signal_runtime_summary_degraded` with bootstrap-aware copy (`Bootstrap: runtime summary stale`) while watch connectivity is healthy — this is expected local scaffolding, not a production outage

That distinction matters during review.

Tip 7: Do not overreact to incomplete polish

Prefer contracts and verify harness first; cosmetic cleanup can wait.

28 What A Good Next Slice Looks Like

A good next slice should:

- keep shared contract semantics stable
- improve one owned behavior
- preserve boot simplicity

- come with verification

Completed thin slices (do not re-do):

- bootstrap + shared contracts + runtime summary
- first watch signal path
- npm workspace dev ergonomics
- first run lifecycle (create → executing → complete)
- startup supervision reliability (`scripts/dev/lib/common.sh`)
- stop/resume, approval, review-ready, SQLite persistence, and briefing-backed dock projections
- workspace list + backend PTY terminal + file-backed Monaco host + nested explorer tree + new-file creation + active-file rename + resizable bottom terminal dock
- richer inbox ranking (severity, recency, unresolved duration, status, action-type, workspace priority)
- split shell regions (`TopBar` , `LeftSidebar` , `CenterWorkbench` , `RightDock` , `StatusBar`) with mockup-shell HUD chrome
- Conversation and Command dock seams backed by control-plane chat endpoints (`POST /api/chat/messages` , `GET /api/workspaces/{workspace_id}/chat/thread` , `GET /api/chat/threads/{thread_id}/history`)

Suggested next slices (2026-07-04):

1. **Coordinator** — KAIRO operator-presence integration when explicitly assigned
2. **Lane B** — agent orchestration hook for chat messages (beyond system ack stub)

Bad next slices:

- broad UI rewrite
- expanding multiple semantic families at once
- changing run-state and signal-state in one uncontrolled pass
- skipping verification because “it is still early”
- claiming full IDE parity when workspace file operations are still intentionally thin-slice (open, edit, save, create, rename) rather than a full VS Code clone

29 Final Guidance

If you are unsure what to do next, choose the smaller move that:

- preserves ownership
- strengthens verification

- reduces placeholders
- keeps the shell boot-safe

That is the design center of this repo right now.

---

30 CI, merge workflow, and worker agents

Updated: 2026-07-22

This chapter explains how Axon-X lands code through GitHub CI, how that relates to the `dev` branch, and how **company employee agents** (the roster in IDE / Mission Control) work — including how to tell whether they are actually running and how good the automation is today.

Related: `docs/CI_GATES.md` , `docs/planning/EXECUTION_PLAN.md` , `recent-operator-features.md`

Names — do not confuse these

NAME	WHAT IT IS
Axon-X Fast Gate	GitHub Actions workflow (`.github/workflows/fast-gate.yml`) — automated tests on every push and PR
GitHub → Agents tab	GitHub product UI (Copilot / cloud agents). **Not** the Axon company roster
Company roster / employee agents	Config-driven roles (`Night Watch` , `Shell Craft` , …) in `config/workspace-agents.json` — continuous workers on a schedule
Lane B / IDE agent	Cursor CLI agent runs you start from the IDE composer (operator-driven)
Cursor worker-server	Internal Cursor subprocess; not an Axon employee by itself

---

CI and merge — how it works

Integration truth

BRANCH	ROLE
`dev`	Integration branch. Protected on GitHub — `fast-gate` **must pass** before merge
`feat/*`	One feature slice per branch (e.g. `feat/operator-console`)
`master`	Legacy default; **not** the day-to-day Axon-X integration target

Day-to-day console work targets `dev` via pull request. Do not camp on `master` for operator-console slices.

What triggers CI

Workflow: **Axon-X Fast Gate** (`.github/workflows/fast-gate.yml`)

Runs on:

- every **push** to any branch
- every **pull_request** (open, sync, reopen)

In the GitHub UI: **Actions** → workflow runs show green ✓ or red ✗. A PR also shows the same check on the PR page.

Poll Fast Gate from the terminal (required after every push)

Agents and operators should **not** wait for someone to notice a red run:

```
./scripts/ops/watch-fast-gate.sh
## or follow a known run id:
./scripts/ops/watch-fast-gate.sh 29925529734
```

On failure:

```
gh run view --log-failed
## Typical Fast Gate early fail: file-size ratchets in
scripts/guardrails/hotspot_budgets.json
```

Axon-X company ownership: Rowan (watcher) + Mira (Lead) own Fast Gate for `workspace_axon_watch`. Quinn (integrations) supports Actions wiring.

What Fast Gate runs (~2–3 minutes)

1. `npm run verify:contracts` — shared types, control-plane/watch contract unit tests, file-size ratchets
2. `npm run verify:hotspot-changes` — critical hotspots must shrink or stay within budget when changed (PR compares against base branch)
3. `npm run verify:console-web` — Vue typecheck, full Vitest suite, production build
4. `scripts/verify/all.py --strict-pending` — import boundaries, DTO budgets, ADR governance, latency scaffold

If any step fails, the run is red and `dev` **cannot merge** that PR until fixed.

Operator merge workflow

```
feat/my-slice → push → open PR to dev → fast-gate green → merge → git pull origin dev
```

Step by step:

1. **Work on a feature branch** (not directly on `dev`).
2. **Commit focused slices** — one concern per PR when possible (worker scheduler separate from IDE chrome, separate from connector parity).
3. **Push** the branch (`git push -u origin feat/my-slice`).
4. **Open a PR** to `dev` (`gh pr create --base dev --head feat/my-slice`).
5. **Wait for** `fast-gate` — green on the PR (not only on an earlier push).
6. **Merge** when green (GitHub UI or `gh pr merge`).
7. **Sync locally:** `git fetch origin dev && git checkout feat/my-slice && git merge origin/dev` (or rebase per team habit).

Important: A green check on an old push does **not** guarantee the open PR is green. Always read the check on the **PR** itself.

Local preflight (before you push)

```
npm run verify:preflight   # via scripts/sc on commit — blocks bad commits locally
npm run verify:contracts   # fastest backend + ratchet signal
cd apps/console-web && npm run typecheck && npm run test
```

Full local bundle (slower):

```
npm run verify
```

Nightly gate (not on every PR)

Workflow: `.github/workflows/nightly-verify.yml` — live stack evidence. See `docs/CI_GATES.md`.

Worker / employee agents — how they work

What they are

Axon-X models each workspace as a **small company** with named employees (lead, watcher, frontend, backend, integrations). Configuration lives in:

- `config/workspace-agents.json` — companies, roles, schedules, `owns` text

- `services/control-plane/app/workspace_agents/` — load config, derive status, scheduler

UI:

- **IDE → TEAM tab** (`IdeTeamPanel` → `CompanyRosterPanel`)
- **Talk / Assign** — pre-fills the IDE composer (operator-driven); does not by itself start a scheduled worker

Schedules

SCHEDULE	MEANING
`on_demand`	No automatic shift (typical for **lead**)
`always_on`	Watcher-style; scheduler starts bounded shifts (e.g. **Night Watch**)
`continuous`	Specialist roles (frontend, backend, integrations) get recurring shifts

Skipped from auto-schedule: `lead` , `overview_agent` .

Scheduler loop (WA-1)

When the control plane starts (`services/control-plane/app/bootstrap.py`):

1. `start_continuous_worker_scheduler()` runs a background tick (default **every 45s**).
2. Each tick, for each `always_on` / `continuous` employee without an active role-tagged run, the scheduler: - `create_run(..., employee_role=<role>)` — run record in SQLite - `dispatch_continuous_worker_run()` — headless Lane B with a role-scoped prompt (`worker_prompt.py` → `worker_dispatch.py` → Cursor/Codex CLI)

Guards (so one restart cannot flood the fleet):

- `MAX_STARTS_PER_TICK` (6) — cap new starts per tick
- `MAX_ACTIVE_EXECUTING` (24) — skip new starts when executing debt is high
- Per-role gate — only one busy shift per `(workspace_id, employee_role)` at a time

Environment knobs (`.env` / vault)

VARIABLE	DEFAULT	PURPOSE
`AXON_WATCH_WORKER_SCHEDULER`	`1`	Enable/disable scheduler
`AXON_WATCH_WORKER_SCHEDULER_DISPATCH`	`1`	Lane B dispatch after `create_run`
`AXON_WATCH_WORKER_SCHEDULER_INTERVAL_SECONDS`	`45`	Tick interval
`AXON_WATCH_WORKER_RUN_STALE_SECONDS`	`720`	Fail/cancel idle role-tagged shifts older than this
`AXON_WATCH_EMPLOYEE_RUN_RETENTION_PER_ROLE`	`8`	Keep this many finished role-tagged runs per workspace/role
`AXON_WATCH_REVIEW_READY_STALE_SECONDS`	`14400`	Auto-complete idle untagged `review_ready` checkpoints older than this

Disable scheduler in tests: `AXON_WATCH_WORKER_SCHEDULER=0`.

Staffing file override: `AXON_WATCH_WORKSPACE_AGENTS_FILE` (path to JSON).

Stale shifts and history retention

Hung role-tagged runs can block the next shift for that role. The control plane:

- Reaps idle employee runs on each scheduler tick and on startup
- Exposes `POST /api/runs/reconcile-stale` for an on-demand reap
- Drains finished employee-run history beyond the per-role retention window on startup / `POST /api/runs/prune-employee-history` (scheduler ticks prune in smaller batches so shifts stay responsive)
- Completes abandoned untagged `review_ready` checkpoints on startup, each scheduler tick, and `POST /api/runs/reconcile-review-ready` so briefing stays honest when review was never dismissed

Approval waits are never auto-completed or pruned. Fresh review checkpoints stay until the idle TTL elapses.

How to know if employee agents are working

1. Company roster (UI)

IDE mode → **TEAM** tab (or operator company panel). Each employee shows a status: `idle`, `watching`, `executing`, `planning`, `verifying`, etc.

Good sign: specialists (`frontend`, `backend`, …) move to **`executing`** when their shift is active, not only the lead mirroring any workspace run.

2. Mission Control / runs API

```
curl -sS http://127.0.0.1:8787/api/runs | python3 -c "
import sys, json
runs = json.load(sys.stdin)
if isinstance(runs, dict): runs = runs.get('runs', runs.get('items', []))
for r in runs:
    if r.get('employee_role') and r.get('phase') not in ('completed','failed','cancelled'):
        print(r['run_id'], r['workspace_id'], r['employee_role'], r['phase'],
r.get('summary','')[:50])
"
```

Good sign:

- `employee_role` is set (`watcher`, `frontend`, …)
- `phase` is `executing` (or progresses to `completed` / `failed` with receipts)
- Summary mentions `continuous worker shift`

Bad sign (phantom worker): `executing` forever with **no** `cursor-agent` process and **no** `runtime_dispatch` receipt on the run history.

3. Live CLI processes

```
pgrep -af 'cursor-agent.*agent --print' | head
```

You should see prompts like *"You are Shell Craft, the frontend employee for workspace workspace_axon_watch…"* when dispatch is active.

4. Control plane health

```
./scripts/dev/check-health.sh
curl -sS http://127.0.0.1:8787/api/health
```

5. Agent shift digests (optional)

When agents write them, receipts live under `docs/ops/agent-reports/` (untracked until committed). Use for human-readable "what this shift did."

6. Unit tests

```
PYTHONPATH=services/control-plane python3 -B -m unittest \
  tests.test_workspace_agent_scheduler \
  tests.test_workspace_worker_prompt \
  tests.test_workspace_agents -q
```

How good are they? (honest assessment)

DIMENSION	TODAY	TARGET
Visibility	Roster + runs show role and phase	Same, plus ranked coaching in briefing (SA-2)
Autonomy	Bounded scheduled shifts with role prompts	Continuous office cadence + checkpoint "meetings"
Continuity	One shift per role; tick every 45s	Stale reaper + handoff memory across shifts
Quality of work	Depends on Lane B + workspace scope; can edit/run in bound repo	Role-scoped playbooks (e.g. DashPro CI triage)
Safety	Debt caps, no auto-start for lead; dispatch uses controlled prompts	Autonomy ladder per workspace (`observe` → `full`)

They are real workers when: Lane B dispatch is on, `cursor-agent` is running with the role prompt, and runs complete or fail with receipts — not when the scheduler only created an `executing` row.

They are not yet: a full autonomous company that manages its own PRs, branches, or SA-2 coaching — that is the next execution slices on `dev`.

Quick reference commands

```
## CI locally
npm run verify:contracts
npm run verify:console-web

## Open PR
gh pr create --base dev --head feat/my-slice --title "..." --body "..."

## PR checks
gh pr checks

## Worker scheduler tests
PYTHONPATH=services/control-plane python3 -B -m unittest
tests.test_workspace_agent_scheduler -q

## Stale reconcile (when loaded on CP)
curl -X POST http://127.0.0.1:8787/api/runs/reconcile-stale
```

PDF: This chapter is bundled into `~/Desktop/Axon-X-How-To-Handbook.pdf` when you run `./scripts/docs/build-howto-handbook-pdf.sh` (rebuild after handbook edits).

Autonomy gates & service identity (operator directives)

Do this in order. Do not skip ahead.

This page is the short, directive companion to `docs/AXON-X-AUTONOMY-MASTER-PLAN.md`. It tells you **what to turn on**, **what to leave off**, and **how to prove it**.

Last updated: 2026-07-22

---

1. Current gate status (what "done" means)

GATE	STATUS	YOU MUST
0–1	Closed	Keep a known-good commit; do not mix unrelated dirty trees into autonomy PRs
2 thin + residuals	Closed	Use `local_token` on any non-loopback surface; set operator + internal tokens
3	Closed	Continuous workers use disposable `worker/<run_id>` worktrees only
4	Closed	Continuous workers only run from a **leased task**; scheduler stays **off** until you intentionally enable it
5	Closed	Use Lead plan/fan-out/replan APIs; keep scheduler off until Gate 6 verification lands

Scheduler rule (non-negotiable for now): keep continuous workers `effective_enabled: false` unless you are deliberately testing Gate 4 with a bounded task board and you are watching the machine.

Check:

```
curl -sS http://127.0.0.1:8787/api/worker-scheduler | jq '{scheduler_enabled,
effective_enabled, executing_count}'
```

Expect `effective_enabled: false` on the daily-driver host.

---

2. Gate 4 — Task ledger (how to use it)

What it is

Every continuous worker shift must bind to one durable task with:

- goal + acceptance criteria
- owner role
- attempt budget

- lease holder + expiry
- terminal outcome (`completed` / `failed` / `cancelled`)

Mission Control shows the board (open / leased / done / failed).

Operator actions

1. **Create a task** (API or Mission Control task board): - `POST /api/workspaces/{workspace_id}/tasks`
2. **Leave the scheduler off** unless you mean to run continuous workers.
3. When a worker claims work, the control plane **leases** the task, creates a run with that `task_id`, and refuses dispatch without a leased task.
4. Long shifts **renew** the lease at dispatch start so mid-run expiry does not silently drop ownership.
5. Dependencies must be `completed` before a dependent task can lease.

Do / Don't

DO	DON'T
Create explicit tasks before enabling continuous workers	Let workers "pick something useful" without a task ID
Keep attempt budgets small (default 3)	Leave infinite retries
Cancel obsolete open tasks when goals change	Leave stale open tasks for the wrong role
Verify with `tests.test_gate4_task_ledger`	Treat Gate 4 as done if only the UI exists

Proof commands:

```
./scripts/dev/python.sh -m unittest tests.test_gate4_task_ledger -q
curl -sS http://127.0.0.1:8787/api/workspaces/workspace_demo/tasks | jq .
```

---

3. Gate 5 — Lead planner (how to use it)

Operator actions

1. Preview a goal as an ordered DAG: `POST /api/workspaces/{workspace_id}/lead/plan`
2. Materialize tasks and dependency-ready specialist runs: `POST /api/workspaces/{workspace_id}/lead/fan-out`
3. When the goal changes, use: `POST /api/workspaces/{workspace_id}/lead/replan` — do not manually leave obsolete tasks open.
4. After all specialist tasks are terminal, call: `POST /api/lead/plans/{plan_id}/synthesize`

Safety rules

- Lead assigns only company-roster specialist roles.
- Exact and parent/child path overlaps cannot lease concurrently.
- Explicit replans stop/cancel obsolete active work and persist receipts.
- “Check with all teammates” creates one task/run per specialist; it never picks one winner.
- Switching IDE tabs changes focus only; sibling thread streams remain alive.
- Keep the continuous scheduler **off**. Gate 5 creates ready runs but does not authorize unattended verification.

Proof:

```
./scripts/dev/python.sh -m unittest \
  tests.test_lead_task_plan \
  tests.test_lead_fan_out \
  tests.test_lead_replan -q
npm test -w @axon-watch/console-web -- --run src/lib/workspace-stream-ui.test.ts
```

---

4. Watch service identity (token + proxy mTLS)

Control-plane talks to watch on `/internal/watch/*`. That path must never be anonymous on a reachable host.

Minimum (required on remote)

Set the **same** shared token on control-plane and watch:

```
## ~/.config/axon-watch/deployment.env
AXON_WATCH_INTERNAL_SERVICE_TOKEN=<long random secret>
AXON_WATCH_REMOTELY_REACHABLE=1   # or a non-loopback AXON_WATCH_PUBLIC_BASE_URL
```

Do: generate with `openssl rand -hex 24`
Don’t: leave the token empty on a tunnel / public URL
Don’t: expose watch port `:8788` on the public internet

When remotely reachable, watch **refuses** mutating internal routes if the token is missing (HTTP 503) or wrong (HTTP 401).

Proxy mTLS capability (deployment proof required)

Axon-X supports proxy-verified client certificates **plus** the shared token. This is not end-to-end proof by itself: the proxy must strip incoming verification headers, verify the certificate, and be the only network path to watch.

1. Mint certs once:

```
./scripts/ops/mint-watch-mtls.sh
## writes ~/.config/axon-watch/mtls/{ca,client,server}.{crt,key}
```

2. Put this in `deployment.env` (both services):

```
AXON_WATCH_MTLS_REQUIRED=1
AXON_WATCH_MTLS_CA_FILE=$HOME/.config/axon-watch/mtls/ca.crt
AXON_WATCH_MTLS_CLIENT_CERT=$HOME/.config/axon-watch/mtls/client.crt
AXON_WATCH_MTLS_CLIENT_KEY=$HOME/.config/axon-watch/mtls/client.key
AXON_WATCH_MTLS_ALLOWED_CN=axon-control-plane
```

3. Configure the reverse proxy in front of watch to verify the client cert and forward:

- `X-SSL-Client-Verify: SUCCESS`
- `X-SSL-Client-S-DN: CN=axon-control-plane`

4. Restart: `axonrestart`

Do: keep token **and** mTLS together
Don't: enable `AXON_WATCH_MTLS_REQUIRED=1` without proxy verify headers (every mutating watch call will 401)
Don't: commit private keys into git

Proof:

```
./scripts/dev/python.sh -m unittest
tests.test_gate2_auth_containment.Gate2WatchInternalTokenTests -q
```

---

5. Control-plane operator auth (reminder)

On any remotely reachable console:

```
AXON_WATCH_AUTH_MODE=local_token
AXON_WATCH_OPERATOR_TOKEN=<strong secret>
AXON_WATCH_AUTH_ALLOW_LOOPBACK=0
```

Remote surfaces **force** `local_token` even if env still says `placeholder`. Full Access / exact-effect on remote also require step-up header `X-Axon-Step-Up` (`full-access` or `exact-effect`).

---

6. Verify before you claim a gate closed

```
npm run verify:contracts
npm run verify:console-web
## then push and confirm Axon-X Fast Gate is green on GitHub
```

Evidence lives under `docs/ops/agent-reports/` and the roll-up log `docs/ops/agent-reports/AUTONOMY-EVIDENCE-LOG.md`.

---

7. What to build next

1. Gate 6 — mandatory verifier contract
2. Keep scheduler off until failed verification blocks completion
3. Do not expand mobile mutation until this page's remote auth + mTLS steps are live on that host

---

42 Recent operator features (Gate 3–5 + console UX)

Updated: 2026-07-22
Audience: operators and agents who need plain-language "what changed and how to use it."

Companion to `docs/HOW-TO-HANDBOOK.md` and `autonomy-gates-and-service-identity.md`.

---

1. Mission Control task board (Gate 4)

Where it is (easy to miss)

Mission Control is the Operator **center panel** — but only in **Grid** view.

Your Brain Graph starfield (VAXON CORE / DashPro orbs) is Operator too; it **replaces** the Mission Control title, Fleet health, and Task board until you leave Graph.

YOU ARE HERE	HOW TO OPEN TASK BOARD
Top nav **OPERATOR** + Brain Graph	Click **Mission Control** in the bottom status chips, **or Mission Control · Task board** in the right Intelligence panel
Operator Grid already	Scroll under **Fleet health** → section **Task board**
Top nav **IDE**	Click **OPERATOR** first, then use Grid / Mission Control as above

On Grid you should see the heading **Mission Control**, then **GRID | BRAIN**, then **Fleet health**, then **Task board**.

What the task board is

A durable **task ledger**. Continuous workers may only start when they **lease** an open task. Chat in the IDE is separate (see “Talk vs do work” below).

How to use it

1. Top nav → **OPERATOR**.
2. Leave Brain Graph: **Mission Control** chip (or Intelligence → **Mission Control · Task board**).
3. Under **Fleet health**, use **Task board**.
4. Create a goal + owner role (e.g. `integrations`).
5. Leave the worker scheduler **off** unless you intentionally want continuous shifts.

Talk vs do work (IDE teammate modes)

In the IDE agent dock, open the mode chip (bottom-right, often **Agent FULL ACCESS**):

MODE	USE WHEN
Ask	Simple conversation — “thanks”, questions, status — **no** edits/tools
Plan	Map steps before changing code
Agent	Do the task — tools, edits, approvals
Debug	Reproduce and fix with evidence
Task board / Lead fan-out	Durable assigned work across specialists (Gate 4–5)

Examples

- “Thank you Dana!” / “I am thinking…” → stay on **Ask** (conversation).
- “Ok proceed with that.” / “Fix the payments race” → **Agent** (task).
- “Check with all sub-agents…” → Lead fan-out (N tasks/runs), not one specialty winner.

Example — seed work for Soren

```
curl -sS -X POST http://127.0.0.1:8787/api/workspaces/workspace_dashpro/tasks \
  -H 'Content-Type: application/json' \
  -d '{
    "goal": "Verify quality-gate heap calc does not OOM CI",
    "acceptance_criteria": "Targeted node-heap tests pass with receipts",
    "owner_role": "integrations",
    "attempt_budget": 2
  }' | jq '{task_id, status, owner_role}'
```

Expect `"status": "open"`. When the scheduler is enabled, that role claims/leases it before `create_run`.

Example — list open tasks

```
curl -sS 'http://127.0.0.1:8787/api/workspaces/workspace_dashpro/tasks?status=open' \
  | jq '.items[] | {task_id, goal, owner_role}'
```

---

2. Concurrent IDE conversation tabs

What changed

Switching conversation tabs **no longer stops** another teammate's live stream. Each IDE thread keeps its own SSE session.

Example

1. Start an agent on **Soren**.
2. Open **Marco** and start (or watch) another run.
3. Switch back to Soren — his stream continues; the busy glow stays on both tabs.

Stop still applies to the **focused** thread only.

---

3. Brain Graph workspace labels + colors

What changed

Every **named workspace** orb shows a label (DashPro included). Orbs get a **muted stable tint** from `workspace_id` so they are easier to spot. Health tones (attention / critical) still override.

Example

Operator → Brain Graph → select DashPro in the left rail → the **DASHPRO** chip appears on its orb. Unlabeled spheres are usually signals/connectors/runs, not workspaces.

---

4. Lead planner + fan-out (Gate 5)

What it is

Dana (Lead) turns one goal + company roster into ordered plan items, **persists** them as leased `workspace_tasks`, and for fan-out opens **one run per ready specialist** (not a single specialty-route winner).

PIECE	MODULE / API
Pure plan	`build_lead_task_plan`
Persist	`persist_lead_task_plan` → Gate 4 ledger
Materialize	`materialize_lead_fan_out` → tasks + ready runs
HTTP	`POST /api/workspaces/{id}/lead/plan` · `.../lead/fan-out`

Continuous **dispatch** (Lane B) is still separate — fan-out creates leased tasks and runs with `lead_fan_out_assigned` receipts; the scheduler stays off.

Examples

Fan-out (all specialists in parallel; path conflicts serialize / defer):

```
Goal: "Check with all sub-agents whether DashPro quality-gate heap calc is safe"
→ tasks for watcher, frontend, backend, integrations
→ N ready runs with receipts (deferred if exclusive_paths overlap)
```

```
curl -sS -X POST "http://127.0.0.1:8787/api/workspaces/workspace_axon_watch/lead/fan-out" \
  -H 'content-type: application/json' \
  -d '{"goal":"Check with all sub-agents whether Gate 5 fan-out is wired"}' \
  | jq '{mode, run_count:(.runs|length), deferred:(.deferred|length), receipt}'
```

Sequential (then-chains → dependency task ids):

```
Goal: "Fix the API quality-gate heap calc then update the Expo confirmation screen"
→ backend task, then frontend task that depends on the backend task_id
```

Specialty route stays single-winner for normal prompts; fan-out phrases return `reason=lead_fan_out` instead of picking one teammate.

Proof:

```
./scripts/dev/python.sh -m unittest tests.test_lead_task_plan tests.test_lead_fan_out -q
```

---

5. After every push — watch Fast Gate

Why

`feat/*` and PRs to `dev` must keep **Axon-X Fast Gate** green. A red push blocks merge and often means file-size ratchets or unit tests.

Operator / agent command

```
## From repo root — polls the latest Fast Gate run for this branch
./scripts/ops/watch-fast-gate.sh
```

Or manually:

```
gh run list --branch "$(git branch --show-current)" --workflow "Axon-X Fast Gate" --limit 3
gh run watch   # follows the newest in-progress run
gh run view --log-failed   # when red
```

Who owns CI for which company

WORKSPACE	WHO WATCHES CI
Axon-X (`workspace_axon_watch`)	**Rowan** (watcher) + **Mira** (Lead triage) — Fast Gate for this repo
DashPro	**Cass** (watcher) + **Soren** (integrations / Actions)
Child workspaces	Watcher role for that company; escalate to Lead

VAXON / the Axon-X watcher should treat **this repository's Fast Gate** as part of runtime health after any push to `origin`.

---

6. Workspace company parity (vs DashPro)

DashPro is not a special UI — every bound workspace uses the same Team panel, Mission Control, and Task board. What differed was **staffing**:

WORKSPACE	COMPANY
DashPro / Axon-X / School	Named companies in `config/workspace-agents.json`
TPS / Young Eagles / Axon Local	Named companies (same shape as DashPro)
Other bound projects	Template staffing with **workspace-stable** names + voices (not Mira clones)

After switching to **TPS** in IDE → Team, you should see **Noor / Blair / Vera / Hugo / Tess**, not Mira/Rowan. Mission Control → Grid always keeps the **selected** workspace on the fleet board even when many projects exist.

---

7. What is still off on purpose

CONTROL	STATE	WHY
Continuous worker scheduler	**Off** by default	Needs leased tasks + Lead/fan-out before unattended shifts
Live-checkout continuous edits	**Forbidden**	Gate 3 isolation — workers use disposable worktrees
Gate 5 full fan-out dispatch	**Partial**	Persist + ready runs land; Lane B auto-dispatch still manual / scheduler off

Check scheduler:

```
curl -sS http://127.0.0.1:8787/api/worker-scheduler \
  | jq '{scheduler_enabled, effective_enabled, executing_count}'
```

VAXON Desktop

Packaged Linux desktop shell (Tauri 2) with frozen Watch + Control Plane sidecars on loopback `:8787`.

Verify

```
npm run verify:desktop
npm run prove:desktop:clean-install
npm run prove:desktop:voice
```

Evidence: `docs/VAXON_DESKTOP_VERIFY_EVIDENCE.md`.

Install

```
npm run build:desktop:linux
sudo apt-get install -y ./apps/console-desktop/src-
tauri/target/release/bundle/deb/VAXON_*.deb
```

Requires Debian-family x86_64 + WebKitGTK 4.1; current sidecars need **glibc ≥ 2.42**. Full first-run / tray / update notes: `docs/VAXON_DESKTOP_VERIFY_EVIDENCE.md` and `docs/DUAL_RUNTIME_STARTUP_CONTRACT.md`.

---

44 Online research: SearXNG and legacy Google

Preferred → fallback order (matches the control-plane research path):

1. **SearXNG** when `AXON_WATCH_SEARXNG_URL` is set (recommended local path)
2. **Google Custom Search** (legacy) when `AXON_WATCH_GOOGLE_CSE_API_KEY` and `AXON_WATCH_GOOGLE_CSE_CX` are set (DashPro aliases `EXPO_PUBLIC_GOOGLE_CSE_*` still work)
3. **DuckDuckGo Instant** as the last resort (often sparse)

Preferred: local SearXNG

Start the local instance and point research at it:

```
./scripts/dev/run-searxng.sh
## then set AXON_WATCH_SEARXNG_URL=http://127.0.0.1:8080 in .env (or vault)
```

`config/searxng/settings.yml` must keep `json` under `search.formats` (SearXNG returns **403** for `format=json` when JSON is disabled). Do not rely on public instances that disable the JSON API. After a successful search, capability / receipts should show `provider: searxng`.

SearXNG Search API: https://docs.searxng.org/dev/search_api.html

Legacy: Google Custom Search 403

If SearXNG is not configured and live search reports **403 — this project does not have access to Custom Search JSON API** (reason often `forbidden`):

1. Open Google Cloud Console → APIs & Services → Library for the project that owns that API key (DashPro's Firebase/Cloud project is often `edudashpro`).
2. Find **Custom Search API** (`customsearch.googleapis.com`) and click **Enable**.
3. In **APIs & Services → Enabled APIs / Dashboard**, confirm Custom Search API is listed as enabled for that same project (not a different Cloud project).
4. Confirm the Programmable Search Engine ID (`cx`) still matches the engine in the Programmable Search control panel.
5. Retry a search, or prefer local SearXNG instead of unblocking Google.

If Enable was already clicked and the 403 persists, Google's docs state the Custom Search JSON API is **closed to new customers** (existing customers only, through 2027). In that case, enabling the library toggle cannot grant access; use SearXNG (`./scripts/dev/run-searxng.sh`) rather than creating a new Google search project for Axon-X.

> Official Google overview (legacy path only): https://developers.google.com/custom-search/v1/overview